

AIM: A* Search and Recursive Best-First Search

- Implement the A* Search algorithm for solving a pathfinding problem.
- Implement the Recursive Best-First Search algorithm for the same problem.
- Compare the performance and effectiveness of both algorithms.

Description:

A*:

```
import heapq
```

```
import matplotlib.pyplot as plt
```

```
import networkx as nx
```

```
graph = {
    'Marve Manor': {'Malad': 2.00},
    'Malad': {'Marve Manor': 2.00, 'Goregaon': 2.00, 'Kandivali': 2.00, 'Kurar': 2.00},
    'Goregaon': {'Malad': 2.00, 'Ram Mandir': 2.00},
    'Ram Mandir': {'Goregaon': 2.00, 'Jogeshwari': 2.00},
    'Jogeshwari': {'Ram Mandir': 2.00, 'Andheri': 2.00},
    'Andheri': {'Jogeshwari': 2.00, 'MVLU College': 1.17},
    'MVLU College': {'Andheri': 1.17, 'Western Express Highway': 2.94},
    'Kandivali': {'Malad': 2.00, 'Borivali': 2.00},
    'Borivali': {'Kandivali': 2.00, 'Dahisar': 4.36},
    'Dahisar': {'Borivali': 4.36},
    'Kurar': {'Malad': 2.00, 'Arrey': 2.00},
    'Arrey': {'Kurar': 2.00, 'Jogeshwari East': 2.00},
    'Jogeshwari East': {'Arrey': 2.00, 'Western Express Highway': 2.00},
    'Western Express Highway': {'Jogeshwari East': 2.00, 'MVLU College': 2.94}
}
```

```
heuristics = {
    'Marve Manor': 11.17,
    'Malad': 9.17,
    'Goregaon': 7.17,
    'Ram Mandir': 5.17,
    'Jogeshwari': 3.17,
    'Andheri': 1.17,
    'MVLU College': 0,
    'Kandivali': 11.17,
    'Borivali': 13.17,
    'Dahisar': 17.53,
    'Kurar': 8.94,
    'Arrey': 6.94,
    'Jogeshwari East': 4.94,
    'Western Express Highway': 2.94
}
```

```

}

node_positions = {
    'Dahisar': (0, 10),
    'Borivali': (0.3, 8),
    'Kandivali': (0.6, 6.5),
    'Marve Manor': (-2, 6),
    'Malad': (0, 5),
    'Goregaon': (0.6, 4),
    'Ram Mandir': (1, 3),
    'Jogeshwari': (1.4, 2),
    'Andheri': (1.8, 1),
    'MVLU College': (2.2, 0),
    'Kurar': (-1.2, 4),
    'Arrey': (-1.2, 3),
    'Jogeshwari East': (-0.8, 2),
    'Western Express Highway': (-0.3, 1)
}

def a_star_search(graph, heuristics, start, goal):
    priority_queue = [(heuristics[start], start, [start], 0)]
    visited = set()

    while priority_queue:
        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue
        visited.add(current)

        if current == goal:
            return path, g_score

        for neighbor, edge_weight in graph[current].items():
            if neighbor not in visited:
                next_g = g_score + edge_weight
                next_f = next_g + heuristics[neighbor]
                heapq.heappush(priority_queue, (next_f, neighbor, path + [neighbor], next_g))

    return None, float('inf')

def route_distance(graph, route):
    total = 0
    for a, b in zip(route, route[1:]):

```

```

    total += graph[a][b]
    return total

route_1 = ['Marve Manor', 'Malad', 'Goregaon', 'Ram Mandir', 'Jogeshwari', 'Andheri', 'MVLU College']
route_2 = ['Marve Manor', 'Malad', 'Kurar', 'Arrey', 'Jogeshwari East', 'Western Express Highway', 'MVLU College']

dist_1 = route_distance(graph, route_1)
dist_2 = route_distance(graph, route_2)

print("Route comparison:")
print(f" Route 1 (via Ram Mandir / Jogeshwari / Andheri):      {' -> '.join(route_1)}")
print(f" Total: {dist_1} km")
print(f" Route 2 (via Kurar / Arrey / Western Express Highway): {' -> '.join(route_2)}")
print(f" Total: {dist_2} km")

if dist_1 < dist_2:
    print(f"\nBest route: Route 1, shorter by {round(dist_2 - dist_1, 2)} km\n")
else:
    print(f"\nBest route: Route 2, shorter by {round(dist_1 - dist_2, 2)} km\n")
optimal_path, total_distance = a_star_search(graph, heuristics, 'Marve Manor', 'MVLU College')
print(f"Optimal Path Discovered: {' -> '.join(optimal_path)}")
print(f"Total Road Distance: {total_distance} km\n")

G = nx.DiGraph()
for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(node, neighbor, weight=weight)

plt.figure(figsize=(9, 10))

path_edges = list(zip(optimal_path, optimal_path[1:]))
normal_edges = [edge for edge in G.edges() if edge not in path_edges]

nx.draw_networkx_nodes(G, node_positions, node_size=2800, node_color='lightblue')
nx.draw_networkx_edges(G, node_positions, edgelist=normal_edges, width=1.5,
edge_color='gray', arrows=True)
nx.draw_networkx_edges(G, node_positions, edgelist=path_edges, width=3.5,
edge_color='darkorange', arrows=True)

node_labels = {node: f"{node}\nh(n)={heuristics[node]}" for node in G.nodes()}
nx.draw_networkx_labels(G, node_positions, labels=node_labels, font_size=8,
font_weight='bold')

```

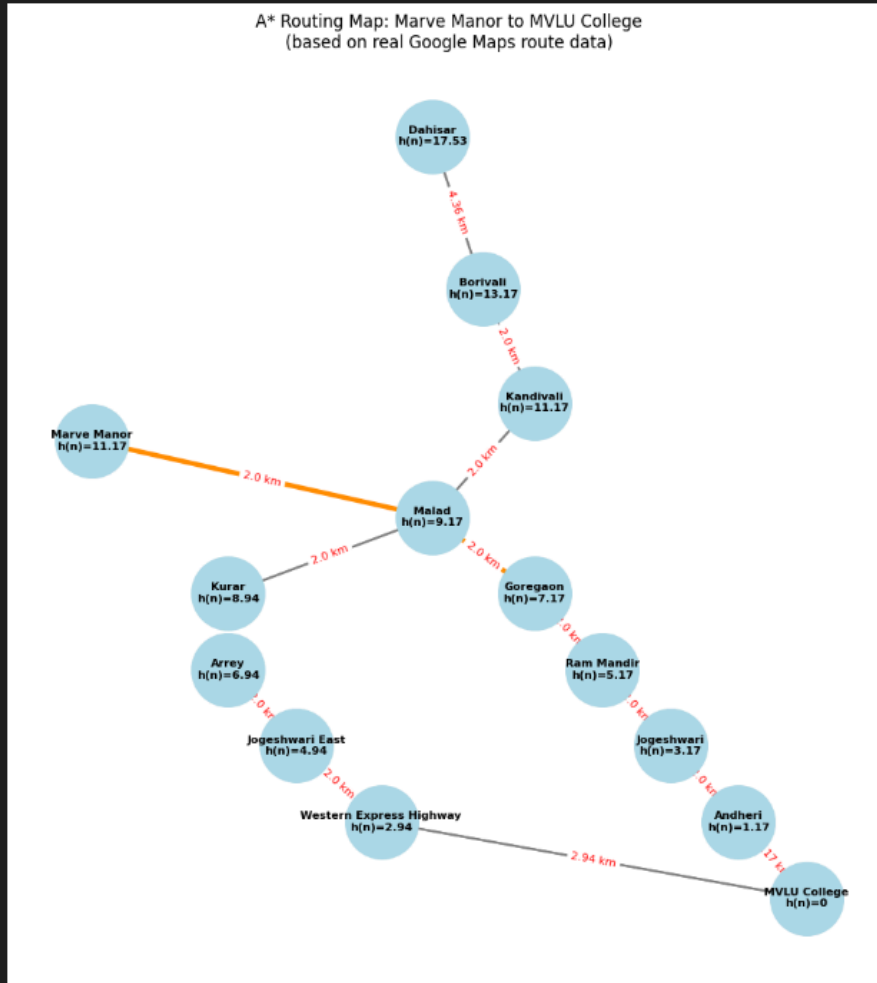
```
edge_labels = nx.get_edge_attributes(G, 'weight')
formatted_edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}
nx.draw_networkx_edge_labels(G, node_positions, edge_labels=formatted_edge_labels,
font_color='red', font_size=8)

plt.title("A* Routing Map: Marve Manor to MVLU College\n(based on real Google Maps route
data)", fontsize=12)
plt.axis('off')
plt.tight_layout()
plt.savefig('/Users/sahil/practice/ai_prac/sem-5/astar_marve_mvlu_map.png', dpi=150,
bbox_inches='tight')
print("Saved plot.")
```

Output:

SHETH L.U.J. AND SIR M.V. COLLEGE

Route comparison:
 Route 1 (via Ram Mandir / Jogeshwari / Andheri): Marve Manor -> Malad -> Goregaon -> Ram Mandir -> Jogeshwari -> Andheri -> MVLU College
 Total: 11.17 km
 Route 2 (via Kurar / Arrey / Western Express Highway): Marve Manor -> Malad -> Kurar -> Arrey -> Jogeshwari East -> Western Express Highway -> MVLU College
 Total: 12.94 km
 Best route: Route 1, shorter by 1.77 km
 Optimal Path Discovered: Marve Manor -> Malad -> Goregaon -> Ram Mandir -> Jogeshwari -> Andheri -> MVLU College
 Total Road Distance: 11.17 km
 Saved plot.



RBFS:

import matplotlib.pyplot as plt

```
graph = {
    'Marve Manor': {'Malad': 2.00},
    'Malad': {'Marve Manor': 2.00, 'Goregaon': 2.00, 'Kandivali': 2.00, 'Kurar': 2.00},
    'Goregaon': {'Malad': 2.00, 'Ram Mandir': 2.00},
    'Ram Mandir': {'Goregaon': 2.00, 'Jogeshwari': 2.00},
    'Jogeshwari': {'Ram Mandir': 2.00, 'Andheri': 2.00},
    'Andheri': {'Jogeshwari': 2.00, 'MVLU College': 1.17},
```

SHETH L.U.J. AND SIR M.V. COLLEGE

```
'MVLU College': {'Andheri': 1.17, 'Western Express Highway': 2.94},  
'Kandivali': {'Malad': 2.00, 'Borivali': 2.00},  
'Borivali': {'Kandivali': 2.00, 'Dahisar': 4.36},  
'Dahisar': {'Borivali': 4.36},  
'Kurar': {'Malad': 2.00, 'Arrey': 2.00},  
'Arrey': {'Kurar': 2.00, 'Jogeshwari East': 2.00},  
'Jogeshwari East': {'Arrey': 2.00, 'Western Express Highway': 2.00},  
'Western Express Highway': {'Jogeshwari East': 2.00, 'MVLU College': 2.94}  
}
```

```
heuristics = {  
    'Marve Manor': 11.17,  
    'Malad': 9.17,  
    'Goregaon': 7.17,  
    'Ram Mandir': 5.17,  
    'Jogeshwari': 3.17,  
    'Andheri': 1.17,  
    'MVLU College': 0,  
    'Kandivali': 11.17,  
    'Borivali': 13.17,  
    'Dahisar': 17.53,  
    'Kurar': 8.94,  
    'Arrey': 6.94,  
    'Jogeshwari East': 4.94,  
    'Western Express Highway': 2.94  
}
```

```
coords = {  
    'Dahisar': (0, 10),  
    'Borivali': (0.3, 8),  
    'Kandivali': (0.6, 6.5),  
    'Marve Manor': (-2, 6),  
    'Malad': (0, 5),  
    'Goregaon': (0.6, 4),  
    'Ram Mandir': (1, 3),  
    'Jogeshwari': (1.4, 2),  
    'Andheri': (1.8, 1),  
    'MVLU College': (2.2, 0),  
    'Kurar': (-1.2, 4),  
    'Arrey': (-1.2, 3),  
    'Jogeshwari East': (-0.8, 2),  
    'Western Express Highway': (-0.3, 1)  
}
```

```

def route_distance(graph, route):
    total = 0
    for a, b in zip(route, route[1:]):
        total += graph[a][b]
    return total

def rbfs_search(start, goal):
    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'), path=[start])
    return path, cost

def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return True, path, g, g

    neighbors = graph[node]
    if not neighbors:
        return False, [], 0, float('inf')

    successors = []
    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            next_g = g + distance
            next_f = max(next_g + heuristics[neighbor], g + heuristics[node])
            successors.append([next_f, neighbor, next_g])

    if not successors:
        return False, [], 0, float('inf')

    while True:
        successors.sort(key=lambda x: x[0])
        best = successors[0]

        if best[0] > f_limit:
            return False, [], 0, best[0]

        alternative_f = successors[1][0] if len(successors) > 1 else float('inf')

        success, result_path, total_g, returned_f = rbfs(
            best[1], goal, best[2], min(f_limit, alternative_f), path + [best[1]]
        )

        best[0] = returned_f

    if success:

```

```

return True, result_path, total_g, returned_f

route_1 = ['Marve Manor', 'Malad', 'Goregaon', 'Ram Mandir', 'Jogeshwari', 'Andheri', 'MVLU
College']
route_2 = ['Marve Manor', 'Malad', 'Kurar', 'Arrey', 'Jogeshwari East', 'Western Express
Highway', 'MVLU College']

dist_1 = route_distance(graph, route_1)
dist_2 = route_distance(graph, route_2)

print("Route comparison:")
print(f" Route 1 (via Ram Mandir / Jogeshwari / Andheri):      {' -> '.join(route_1)}")
print(f" Total: {dist_1} km")
print(f" Route 2 (via Kurar / Arrey / Western Express Highway): {' -> '.join(route_2)}")
print(f" Total: {dist_2} km")

if dist_1 < dist_2:
    print(f"\nBest fixed route: Route 1, shorter by {round(dist_2 - dist_1, 2)} km\n")
else:
    print(f"\nBest fixed route: Route 2, shorter by {round(dist_1 - dist_2, 2)} km\n")

path, total_dist = rbfs_search('Marve Manor', 'MVLU College')
print(f"Optimal RBFS Path: {' -> '.join(path)}")
print(f"Total Road Distance: {total_dist} km\n")

plt.figure(figsize=(10, 11))

for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]
        is_path = node in path and neighbor in path and path.index(neighbor) == path.index(node)
+ 1
        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)
        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width, zorder=1)
        plt.text((x1 + x2) / 2, (y1 + y2) / 2, f"{dist}km", color='red', fontsize=9, ha='center',
clip_on=False)

for node, (x, y) in coords.items():
    color = '#f1c40f' if node in path else '#3498db'
    plt.scatter(x, y, color=color, s=1400, zorder=2, clip_on=False)
    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center', va='center', color='white', fontsize=7,
fontweight='bold', clip_on=False)

```

```
all_x = [c[0] for c in coords.values()]
all_y = [c[1] for c in coords.values()]
plt.xlim(min(all_x) - 1.5, max(all_x) + 1.5)
plt.ylim(min(all_y) - 1, max(all_y) + 1)

plt.title(f"RBFS Route Map: Marve Manor to MVLU College (Total: {total_dist}km)", fontsize=12,
fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.savefig('/Users/sahil/practice/ai_prac/sem-5/rbfs_marve_mvlu_map.png', dpi=150,
bbox_inches='tight')
print("Saved plot.")
```

Output:

SHETH L.U.J. AND SIR M.V. COLLEGE

Route comparison:

Route 1 (via Ram Mandir / Jogeshwari / Andheri): Marve Manor -> Malad -> Goregaon -> Ram Mandir -> Jogeshwari -> Andheri -> MVLU College
Total: 11.17 km
Route 2 (via Kurar / Arrey / Western Express Highway): Marve Manor -> Malad -> Kurar -> Arrey -> Jogeshwari East -> Western Express Highway -> MVLU College
Total: 12.94 km

Best fixed route: Route 1, shorter by 1.77 km

Optimal RBFS Path: Marve Manor -> Malad -> Goregaon -> Ram Mandir -> Jogeshwari -> Andheri -> MVLU College
Total Road Distance: 11.17 km

Saved plot.

RBFS Route Map: Marve Manor to MVLU College (Total: 11.17km)

